

MSc Data Science Project

7PAM2002-0901-2024

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

Uncertainty-Aware Deep Learning for Robust Chest X-ray Interpretation

Student Name and SRN:

DEEPIKA MOKKAPATI
22075704

Supervisor: Alyssa Drake

Date Submitted: 05-01-2024

Word Count: 7485

GitHub address: <https://github.com/Deepikamokkapati>

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in Data Science at the University of Hertfordshire.

I have read the guidance to students on academic integrity, misconduct and plagiarism information at [Assessment Offences and Academic Misconduct](#) and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project module or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6).

I have not used chatGPT, or any other generative AI tool, to write the report or code (other than where declared or referenced).

I did not use human participants or undertake a survey in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student SRN number: 22075704

Student Name printed: DEEPIKA MOKKAPATI

Student signature:



UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Abstract

With an emphasis on comparing pre-trained models, including EfficientNetB0 and DenseNet121, with a bespoke Convolutional Neural Network (CNN) generated from scratch, this study investigates the use of deep learning methods for robust interpretation of chest X-rays. The main goal was to assess the models' capacity to identify many chest diseases and handle the intrinsic difficulties presented by medical imaging datasets such as class imbalance and strong inter-class similarity. Pre-trained models were beneficial in using transfer learning from big, varied datasets, according to results showing they beat bespoke CNN in accuracy and Area Under the Curve (AUC) measures. Though they failed with others like pleural effusion and consolidation, EfficientNetB0 and DenseNet121 showed moderate classification performance with strengths in identifying certain disorders like cardiomegaly. With a much-reduced accuracy, the custom CNN drew attention to how poorly limited datasets allow one to train from scratch. Notwithstanding the improvements, all models struggled generally with remembering for crucial lessons.

Table of Contents

1. Introduction	1
1.1. Overview of Chest X-ray Imaging	1
1.2. Challenges in Manual Interpretation of Chest X-rays.....	2
1.3. Advantages of Deep Learning in Medical Imaging	3
1.4. Research Aim.....	4
1.5. Research Questions	4
1.6. Project Objectives.....	4
1.7. Ethical Considerations	5
2. Literature Review	6
2.1. Advances in Uncertainty Quantification for Medical Imaging	6
2.2. Key Insights.....	9
3. Methodology.....	11
3.1. Dataset Collection	11
3.2. Data Loading.....	12
3.3. Data Pre-processing.....	13
3.3.1. Label Extraction and Transformation.....	13
3.3.2. Image Pre-processing and Augmentation	13
3.4. Deep Learning Modelling	13
3.4.1. EfficientNetB0	14
3.4.2. DenseNet121	15
3.4.2. Individual CNN	16
3.5. Model Evaluation.....	17
4. Discussion.....	18
4.1. EfficientNetB0 Architecture Design	18
4.2. DenseNet121 Architecture Design	19

4.3. Custom CNN Model Architecture Design	20
5. Experimental Results.....	22
5.1. EfficientNetB0 Results	22
5.2. DenseNet121 Results	23
5.3. Custom CNN Results.....	24
6. Conclusion	26
6.1. Performance of Pre-trained and Custom Models	26
6.2. Future Directions.....	27
References.....	28
Appendices.....	31

1. Introduction

1.1. Overview of Chest X-ray Imaging

Among the most often used diagnostic instruments in contemporary medicine, chest X-rays provide a non-invasive and quick approach to evaluate different diseases within the chest cavity. Low-dose ionising radiation is used in the imaging process to provide images of the thoracic structures—that is, the lungs, heart, blood vessels, airways, and chest and spine bones. Chest X-rays are the pillar of diagnostic radiology as these images help to spot anomalies and direct clinical choices (Lakhani, 2017). One cannot stress the part chest X-rays play in medical diagnosis. Because of their accessibility, speed, and economy, they are first screening tool of choice in many healthcare environments. Chest X-rays are a common tool used by doctors to find and track lung and surrounding structural illnesses. For instance, considering symptoms such chronic coughing, chest discomfort, dyspnoea, or fever—which can point to underlying medical problems—you absolutely must get chest X-rays. They also are very important in determining how well treatments—such as medicines for lung infections or tracking healing after surgery—work (Sharma et al., 2020).



Figure 1 - Chest X-Ray Sample

In many common disorders, chest X-rays are very useful for diagnosis. Of them, pneumonia is the most often diagnosed illness seen on X-rays. The images enable quick diagnosis and therapy by helping one see inflammation or fluid collection within the lung tissues. Likewise, chest X-rays—which show distinctive lesions or cavities—are often used in diagnosis of tuberculosis (TB), a bacterial illness mostly affecting the lungs. The COVID-19 epidemic marks

yet another important use for chest X-rays. Ground-glass opacities and bilateral lung infiltrates are among the hallmarks of severe acute respiratory syndrome coronavirus 2 (SARS-CoV-2) infection identified using radiographic imaging. Other pulmonary disorders include pleural effusion, lung cancer, and COPD also depend critically on chest X-rays. They are also used to find fractures or anomalies in the ribs and spine and to assess cardiac hypertrophy, which might point to heart failure (Giełczyk et al., 2022).

1.2. Challenges in Manual Interpretation of Chest X-rays

Manual chest X-ray interpretation calls for expertise, accuracy, and experience; it is a vital but difficult task. From these images, radiologists are essential in detecting diseases; nonetheless, various elements add to the complexity of this procedure. The dependence on human interpretation brings possible unpredictability, so in certain circumstances reliable and consistent diagnosis becomes challenging. One important difficulty is the intricacy of certain anomalies. Many diseases, like minor pneumothorax or early-stage lung cancer, show up on a chest X-ray as mild or absent alterations. Finding these little irregularities requires for great vision and plenty of expertise. Sometimes even seasoned radiologists ignore these results, particularly under time constraint or in large patient loads. Delayed or missed diagnosis might have major effects, which emphasises the need of correct interpretation (Madaan et al., 2021).

The intersection of normal and diseased features presents even another problem. Multiple anatomical features seen in the chest cavity—including the lungs, heart, and bones—can superimpose on one another in a two-dimensional image. Often, this superimposition makes it difficult to distinguish normal fluctuations from actual disease alterations. False-positive results might arise from shadows from overlapping ribs or blood arteries, for instance, resembling the look of a lesion. One more noteworthy difficulty is inter-reader variability. Depending on their training, experience, and subjective judgement, various radiologists will see the same chest X-ray differently. Particularly in uncertain circumstances, this diversity might result in different diagnosis and treatment recommendations. Attempts to standardise interpretations by rules and procedures only help to partly solve this problem as individual variations in perspective remain (Baltruschat et al., 2019).

In addition, time-consuming is manual interpretation, particularly in environments lacking radiological capabilities. Radiologists working in highly sought-after fields might have to evaluate several images per day, which raises their chances of mistakes connected to tiredness. In areas lacking skilled experts, the load is even more heavy, delaying diagnosis and patient treatment (Chen et al., 2020). At last, the existence of confusing elements accentuates the difficulty of interpretation. Variables linked to patients, including age, comorbidities, or past operations, might affect the look of chest X-rays and confound diagnosis calls. For example, past scarring from surgery or TB may seem to be ongoing illness, which would cause diagnostic ambiguity (Guan and Huang, 2020).

1.3. Advantages of Deep Learning in Medical Imaging

Offering major benefits over conventional imaging methods, deep learning has transformed medical imaging. Conventional techniques mostly depend on handmade elements and rule-based algorithms, which often find it difficult to depict the intricacy of medical images. On the other hand, deep learning greatly improves diagnosis performance by automatically learning complex patterns and characteristics using big datasets and neural network designs.

Deep learning has one main advantage in that it can raise sensitivity and accuracy in anomaly detection. Medical images may be analysed by neural networks with a degree of accuracy matching or even exceeding human ability. For instance, convolutional neural networks (CNNs) are rather good in spotting minor anomalies, including early-stage tumours or weak lung lesions, which would be overlooked using conventional techniques. Adding to this, providing efficiency and scalability is deep learning. Ideal for high-volume screening programs once trained, models can examine thousands of images in a fraction of the time it would take a human expert. This efficiency speeds patient diagnosis and treatment as well as lessens professional burden. Strong handling of many imaging situations is another important benefit. Deep learning methods guarantee constant performance across several datasets by adjusting to variances in image quality, resolution, and patient anatomy. To provide a more complete analysis, these models may also combine multimodal data—that is, imaging with clinical information (Bacellar et al., 2021).

1.4. Research Aim

The present study aims to evaluate and compare, using chest X-ray images, pre-trained deep learning models like ResNet50 and DenseNet121 with models generated from scratch in the identification of chest diseases. Particularly in situations with limited training data, the research aims to ascertain the effect of transfer learning on model performance by using a sample from the CheXpert collection. Analysed will be key performance characteristics including accuracy, sensitivity, and robustness to provide understanding of the benefits of using pre-trained models. With the end goal of assisting clinical decision-making by improving the dependability and efficacy of automated chest X-ray interpretation in radiology, the project also intends to investigate how these models recognise certain chest ailments.

1.5. Research Questions

How does the performance of pre-trained models (e.g., EfficientNet, DenseNet121) compare to models trained from scratch in detecting chest pathologies?

- Alternate Hypothesis (H1): Pre-trained models will outperform models trained from scratch in detecting chest pathologies, especially when using smaller datasets, due to the transfer of learned features from large, diverse datasets.
- Null Hypothesis (H0): No significant difference exists in the performance of pre-trained models and models trained from scratch in detecting chest pathologies, regardless of the dataset size.

1.6. Project Objectives

1. Utilize a sample from the CheXpert dataset to ensure feasibility while maintaining relevance in results.
2. Analyse the models depending on important performance criteria.
3. Especially in cases of low training data, find how transfer learning affects model performance.
4. Examine the models' sensitivity to identify certain chest diseases.
5. By means of better model performance in chest X-ray interpretation, provide results that can assist clinical decision-making in radiology.

1.7. Ethical Considerations

Using the anonymised chest radiograph CheXpert dataset, this study follows rigorous ethical standards. The anonymising guarantees that no personal information may be connected to particular individuals, therefore conforming with the General Data Protection Regulation (GDPR) criteria for data protection and privacy. Protecting patient anonymity helps the study to maintain ethical norms fundamental in medical research. In addition, in line with the ethical standards set by the University of Hertfordshire is the study. The conditions on the CheXpert website clearly enable academic usage of publicly available data. This guarantees that the research stays within moral limits and helps to further science. Also, the thorough data collecting techniques used by the Stanford ML Group ensure the integrity of the dataset. Their respect of accepted ethical guidelines guarantees the authenticity and dependability of the dataset for scholarly and scientific projects. Using such a dataset guarantees adherence to accepted ethical and legal norms, therefore reducing the risks connected with patient confidentiality or data abuse.

The ethical approach of the work underlines the responsible use of publicly accessible, anonymised datasets to progress knowledge in chest X-ray interpretation while preserving strong attention to privacy and ethical issues.

2. Literature Review

2.1. Advances in Uncertainty Quantification for Medical Imaging

Gour et al., introduces UA-ConvNet, an uncertainty-aware convolutional neural network model designed to detect COVID-19 from chest X-ray (CXR) images while also quantifying prediction uncertainty. The model leverages EfficientNet-B3 with Monte Carlo (MC) dropout to generate a posterior predictive distribution through multiple forward passes, providing both mean prediction and uncertainty estimates. The model's performance is validated on three datasets, including COVID19Cxr and Kaggle, demonstrating impressive accuracy and sensitivity across binary and multi-class classification tasks. Specifically, UA-ConvNet achieves a G-mean of 98.02% in multi-class classification and 99.16% in binary classification, with high confidence intervals. By incorporating uncertainty estimation, UA-ConvNet enhances diagnostic reliability, underscoring its potential as a robust tool for COVID-19 screening in clinical settings (Gour & Jain, 2022).

Zahari et al., introduces an uncertainty-aware framework for distinguishing benign from malignant nodules in 3D CT images, focusing on enhancing model reliability through uncertainty quantification (UQ). The framework consists of three phases: data preprocessing and model evaluation, UQ, and uncertainty-based data referral. Three UQ techniques were employed—Monte Carlo Dropout (MCD), Deep Ensemble (DE), and Ensemble Monte Carlo Dropout (EMCD)—to assess predictive confidence across several deep learning models, including ResNet, DenseNet, and Inception variants. InceptionResNetV2 achieved the highest average F1 score of 0.845. MCD demonstrated robust overall performance in uncertainty estimation, while DE and EMCD showed strengths in identifying uncertain predictions, valuable for minimizing errors in medical imaging. By setting a data referral threshold, the framework further increased classification accuracy to 95.9%, underscoring the importance of UQ in enhancing diagnostic safety and precision (Zahari et al., 2024).

Shamsi et al., presents a deep learning framework for COVID-19 detection using transfer learning and uncertainty quantification on medical images, employing popular CNN architectures such as VGG16, ResNet50, DenseNet121, and InceptionResNetV2. The framework extracts feature representations from chest X-ray and CT images, which are then analysed by various machine learning methods to identify COVID-19 cases. By calculating

epistemic uncertainty, the study addresses regions where models lack confidence, thereby tackling the out-of-distribution issue. Simulation results show that linear support vector machines and neural networks achieved high accuracy, sensitivity, specificity, and AUC for both image types. Notably, uncertainty levels were higher for CT images than for X-rays, highlighting CT data's greater variability and complexity in COVID-19 diagnosis (Shamsi et al., 2021).

Najdenkoska et al., presents a novel probabilistic approach, variational topic inference, aimed at automating chest X-ray report generation. By incorporating a probabilistic latent variable model, the approach introduces a unique latent variable that corresponds to a specific topic, facilitating topic-based sentence generation within a report. The framework aligns visual and language modalities through conditional variational inference, allowing for the generation of coherent descriptions based on distinct image regions. Additionally, a visual attention module helps the model focus on specific areas within chest X-rays, enhancing the relevance of the generated descriptions. Evaluated on the Indiana U. Chest X-rays and MIMIC-CXR datasets, this method produces reports with diverse sentence structures, not simply replicating training data, while maintaining performance on par with leading models across standard language generation metrics. This innovation represents a promising advancement in radiological report automation (Najdenkoska et al., 2022).

Dong et al., introduces a deep learning model, RCoNet, for COVID-19 detection in chest X-ray images, designed to enhance robustness and performance under noisy conditions. RCoNet integrates several advanced components: Deformable Mutual Information Maximization (DeIM), Mixed High-order Moment Feature (MHMF) extraction, and Multiexpert Uncertainty-aware Learning (MUL). DeIM optimizes the mutual information between input and latent features, enabling more compact and disentangled representations that effectively capture key image characteristics. MHMF leverages high-order statistical features to distinguish complex patterns in medical images, improving the model's discriminative power. Additionally, MUL employs multiple dropout networks to assess prediction uncertainty, mitigating the impact of data noise on performance. Experimental results on the COVIDx dataset demonstrate that RCoNet outperforms other models across various metrics, underscoring its effectiveness and resilience in handling noisy data, which is essential in clinical applications (Dong et al., 2021).

Reddy et al., proposes an uncertainty-aware convolutional neural network (UACNN) for automatic detection of COVID-19 from chest X-ray (CXR) images. The model is built on an EfficientNetB3 architecture, fine-tuned with CXR images and enhanced using Monte Carlo (MC) dropout for uncertainty estimation. The model generates a posterior predictive distribution through multiple forward passes, and the uncertainty is quantified by calculating the mean and entropy of this distribution. The proposed method is evaluated on three datasets: COVID19CXr, XR-images, and Kaggle CXR. With a multi-class classification accuracy of 99.81% on the COVID19CXr dataset, the UACNN method outperforms existing approaches, showcasing its potential for reliable and uncertainty-aware COVID-19 diagnosis using CXR images (Reddy et al., 2023).

Yang et al., introduces an Uncertainty-Aware Attention Mechanism (UAAM) to improve lung nodule segmentation by addressing uncertainty in multiple annotations. The approach uses a Multi-Confidence Mask (MCM), combining Low-Confidence (LC) and High-Confidence (HC) Masks to indicate regions with varying segmentation certainty. The proposed Uncertainty-Guide Multi-Confidence Segmentation Network (UGMCS-Net) consists of three modules: a Feature Extracting Module for lung nodule features, an Uncertainty-Aware Module to produce features based on annotation union, intersection, and set, and an Intersection-Union Constraining Module to refine segmentation predictions using feature distances. The method is evaluated using the LIDC-IDRI dataset, focusing on complex lung nodules. Experimental results show that UGMCS-Net significantly outperforms conventional methods in segmenting difficult nodules, demonstrating its potential in uncertainty-aware medical image segmentation (Yang et al., 2024).

Cheginini et al., presents an uncertainty-aware deep learning model that uses Monte Carlo Dropout for uncertainty quantification. Unlike many existing approaches, the proposed model is designed to classify images from different modalities, specifically mammograms and ultrasounds. The model achieved impressive accuracy rates, including 99.95% on the DDSM mammogram dataset, 98.40% on the BUSM ultrasound dataset, and 91.34% on the BUSI ultrasound dataset. Additionally, the study highlights that the model's high uncertainty in misclassifications can be leveraged to identify areas requiring further inspection. By focusing on uncertain predictions, the model can guide clinicians in refining diagnoses and potentially

improving performance, demonstrating the value of uncertainty estimation in medical image classification (Chegini & Mahlooji Far, 2023).

Aldhahi et al., introduces a novel approach for classifying COVID-19 chest X-rays using deep learning models and an uncertainty-based ensemble voting policy. The method achieves 99% accuracy in distinguishing COVID-19 from normal and pneumonia-related infections. The training strategy incorporates cyclic cosine annealing, cross-validation, and uncertainty quantification through prediction interval coverage probability (PICP), which is used to adjust ensemble voting weights. Additionally, the study presents the Uncertain-CAM technique, enhancing deep learning model explainability and ensuring more reliable COVID-19 classification. The approach also introduces a new image processing method for measuring explainability based on ground truth, comparing it with the widely used Grad-CAM method. This work highlights the integration of uncertainty quantification and explainability for improved medical image classification (Aldhahi & Sull, 2023).

Zhan et al., explores lung nodule classification using two datasets: the LUNA16 dataset and a private dataset containing 641 nodules with 454 annotated with six key signs by radiologists. The proposed system consists of a 3D CNN model and a semi-supervised learning (SSL) method called the uncertainty-aware self-training framework with consistency regularization (USC). The system uses supervised learning with labelled data and an uncertainty-based strategy to select pseudo-labelled samples for unsupervised training, enhancing model optimization. Experimental results show that the 3D CNN model achieved a mean average precision (mAP) of 0.870 and AUC of 0.782. The USC method significantly improved performance, increasing mAP by 7.6% and AUC by 8.1%, demonstrating its effectiveness in leveraging unlabelled data for lung nodule sign classification (Zhan et al., 2023).

2.2. Key Insights

Particularly with regard to chest X-ray (CXR) interpretation, the literature review emphasises numerous important new ideas on uncertainty-aware deep learning models in medical imaging. One important realisation is the increasing relevance of uncertainty estimates in improving the interpretability and dependability of deep learning models used for medical diagnosis. Often used techniques include Monte Carlo dropout, Deep Ensembles, and Uncertainty-Aware Attention Mechanisms to estimate both aleatoric (data-related) and

epistemic (model-related) uncertainties, so enabling more accurate predictions and better decision-making under uncertain circumstances.

The examined studies show how uncertainty-aware models increase diagnostic accuracy and sensitivity in identifying illnesses such as COVID-19, pneumonia, and lung cancer using chest X-rays. Especially, uncertainty quantification helps models to highlight doubtful forecasts, hence pointing doctors to areas requiring further investigation. In medical imaging, where high-stakes choices are often made depending on automated systems, this is very vital. Moreover, various research highlights how semi-supervised learning and transfer learning help to manage insufficient labelled data, a typical difficulty in medical image analysis. By effectively using both labelled and unlabelled data, techniques such as uncertainty-aware self-training framework and cross-task supervising help to enhance model performance. Finally, the research emphasises how well uncertainty-aware deep learning models might improve the clinical applicability, explainability, and resilience of chest X-ray interpretation systems, hence increasing their dependability in practical healthcare environments.

3. Methodology

The objective of this work is to develop a deep learning model capable of multiple category classification of chest X-ray images depending on the existence of different illnesses. One may separate the approach into main phases. Development of an efficient model that can effectively identify chest X-ray images and generalise well to unknown data depends on each of these steps. The approach is thoroughly described in this chapter.

3.1. Dataset Collection

Comprising a complete collection of chest X-rays intended to support the creation of automated algorithms for radiological image interpretation, the CheXpert dataset offers Designed by the *Stanford ML Group*, it offers a large-scale dataset with expert-labeled annotations for many chest illnesses, therefore addressing difficulties in medical image analysis. Comprising 224,316 chest radiographs from 65,240 individuals, the CheXpert Small dataset is a downsampled variant of the original CheXpert dataset including both frontal and lateral views of X-rays.

The dataset guarantees a wide representation of several health issues by including varied patient demographics and being gathered from many institutions. This large collection of clinical disorders—cardiomegaly, pneumonia, atelectasis, pleural effusion—offers great information for training deep learning models to identify and categorise these disorders. Radiologists label every X-ray in the dataset one or more conditions; they additionally designate uncertainty labels to show the diagnosis's confidence. The CheXpert dataset has a special characteristic in that it includes uncertainty annotations—that is, images in which the radiologist's diagnosis is uncertain or ambiguous. These annotations provide further information that can enable models to manage uncertainty—a typical difficulty in medical imaging. Additionally included in the dataset is a reference standard used to assess automated models' performance against human radiologists.

Advancement of medical image analysis depends much on the CheXpert dataset, which comprises both small and original versions. It enables the creation of AI-driven diagnostic tools that, by prioritising cases, assisting clinical decision-making, and maybe worldwide improvement of healthcare outcomes, may help healthcare providers.

Original Source: <https://stanfordmlgroup.github.io/competitions/chexpert/>

Data collection source: <https://www.kaggle.com/datasets/ashery/chexpert>

3.2. Data Loading

This research uses the CheXpert dataset, a large-scale collection of chest X-ray images tagged with 14 unique discoveries. Each item in the dataset, which is kept in CSV files, relates to a chest X-ray image along with labels showing the presence or absence of certain diseases including pleural effusion, cardiomegaly, and pneumonia. Any machine learning project starts with data loading and is absolutely vital. There are many folders and CSV files in the dataset; initially, one loads the training and validation data from the provided CSV files. Reading the CSV files—which provide information including the path to every image and the matching labels—helps one do this.

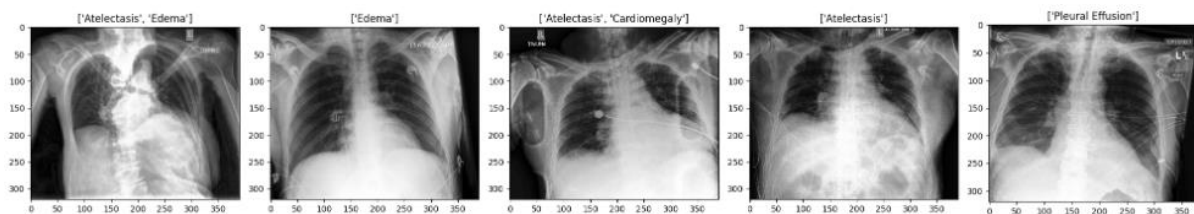


Figure 2 - Images from dataset

Extracted from the information, paths to the images are found; each image is connected with its label depending on the circumstances it shows. This dataset uses a multi-label label format; hence a single image may be connected with many conditions. These labels are converted into a format suitable for model training.

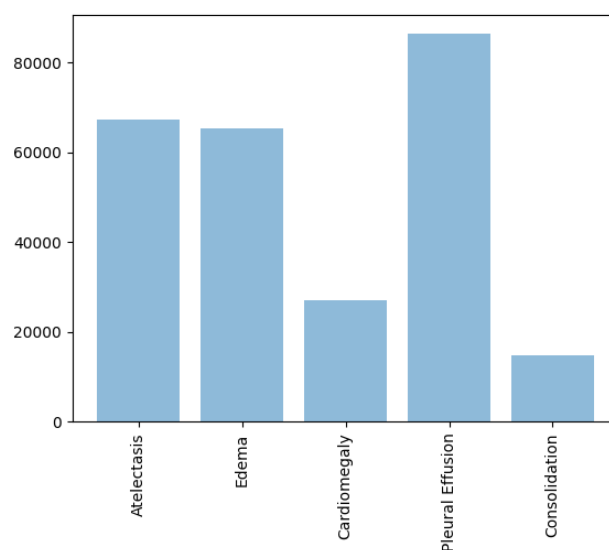


Figure 3 - Class labels distribution

3.3. Data Pre-processing

Data preparation comes next after the data is loaded. This stage guarantees that the data is organised and cleaned suitably for the model. Among the various important pre-processing tasks are image augmentation, label extraction, and management of missing data.

3.3.1. Label Extraction and Transformation

Often recorded in a multi-class format, the labels in the CheXpert dataset each relate to a condition and each row relates to a image. Certain labels can be absent or shown as a peculiar value (-1), meaning the label is unidentified. During the data preparation stage, any photos with missing labels are excluded handling the missing labels. This guarantees that the model receives only training on correct labels from data. Each label is then converted into a binary format wherein it is either 0 (absence) or 1 (presence). The model must undergo this change if it is to grasp whether any condition exists or not.

3.3.2. Image Pre-processing and Augmentation

Resizing photos to a predetermined size fit for the neural network marks the pre-processing stage for image data. The photographs in this project are 75x75 pixel downsized. Additionally used to standardise the photos are image normalising methods like scaling pixel values between 0 and 1. This is important as deep learning models benefit from same range of scaled input features—in this example, pixel values.

Another crucial pre-processing technique that artificially increases the variety of the training data therefore enhancing the generalising capacity of the model is data augmentation. Common augmentation methods include zooming, horizontal flips, width and height changes, and random rotations. These augmentations increase the model's resilience in practical uses and enable it to become invariant to small modifications.

3.4. Deep Learning Modelling

Particularly in medical imaging, the evolution of deep learning models for image classification tasks depends on sophisticated Convolutional Neural Networks (CNNs) able to effectively extract relevant patterns from intricate image data. Three well-known models—Efficient Net B0, DenseNet 121, and custom CNNs—are often discussed in this setting. Every one of these designs has special qualities that fit various image classification tasks, including medical image analysis involving diagnosis of chest X-rays.

3.4.1. EfficientNetB0

Part of the EfficientNet family of models, EfficientNet B0 was developed to solve the problem of effectively scaling deep learning configurations. The idea of compound scaling drives much of EfficientNet's innovation. EfficientNetB0 adjusts the width, depth, and resolution of the network in a balanced manner to get maximum performance with lowest computing cost instead of arbitrarily raising these parameters.

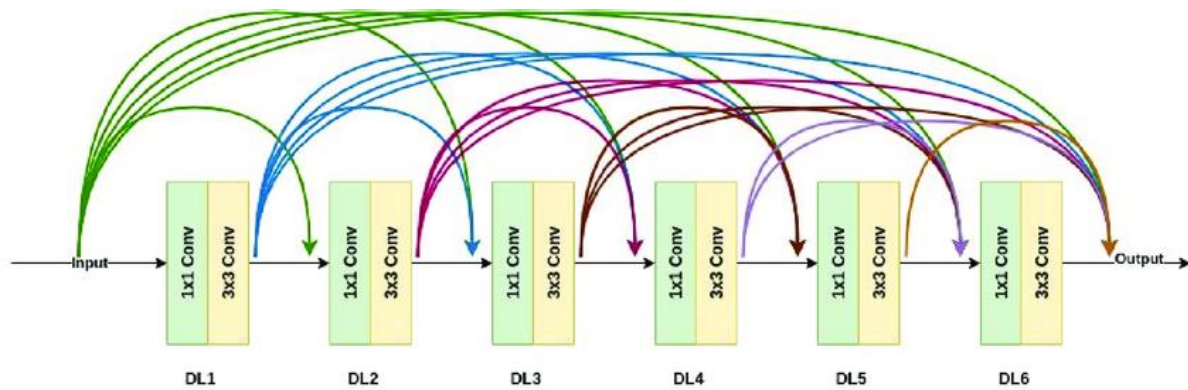


Figure 4 - Architecture of EfficientNetB0

EfficientNetB0 starts with a baseline neural network including many blocks of convolutional layers in design. The scaling method guarantees that the network is not over-parameterized, therefore optimising computing resources as well as model accuracy. EfficientNetB0's high accuracy-to-parameter ratio made possible by this balanced scaling makes it especially appropriate for jobs where computational efficiency is vital, including those involving model deployment in medical situations with restricted hardware resources. Usually trained using the ImageNet dataset—which comprises millions of images across a broad spectrum of categories—EfficientNetB0 is a pre-trained model. Through transfer learning, this pre-training lets EfficientNetB0 acquire generic characteristics like edges, textures, and patterns that may be applied to various tasks. The model's low-level feature-capturing foundation layers—which are frozen for medical imaging tasks like chest X-ray classification—usually freeze their weights during fine-tuning. Only the highest level of classification layers is trained to fit the particular dataset, therefore enabling quicker training and less need for a lot of labelled medical data. Maintaining excellent performance while controlling model complexity is made possible by the compound scaling method used in EfficientNetB0. This enables EfficientNetB0 to get state-of-the-art results with less parameters than conventional CNN designs, including ResNet and VGG, thus attracting for real-world applications.

3.4.2. DenseNet121

By adding dense connections between layers, DenseNet121—also known as Densely Connected Convolutional Networks—another potent CNN design seeks to surpass conventional deep learning models. Usually in a conventional CNN, every layer gets inputs just from the one before it. DenseNet121, on the other hand, expands on the concept that every layer should have input from all past levels, hence producing dense connections. This facilitates simpler training of deeper networks via greater gradient flow and better feature reuse.

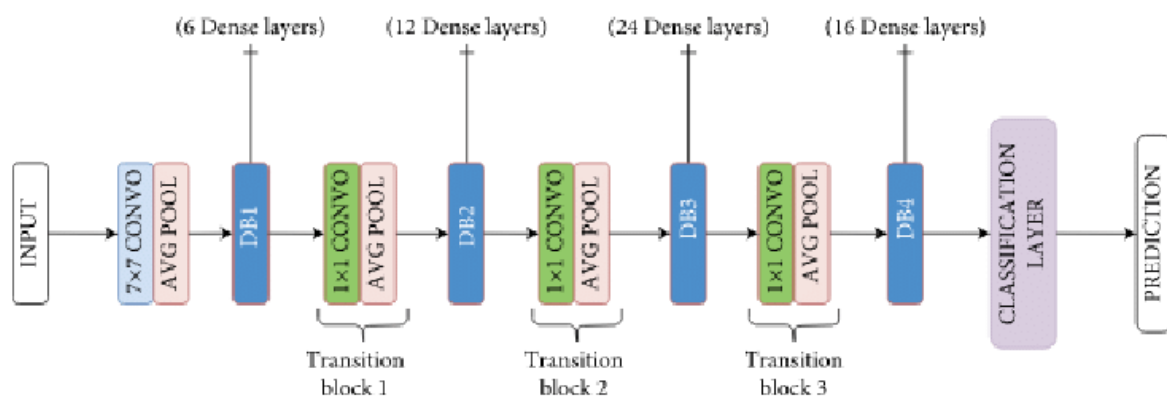


Figure 5 - Architecture of DenseNet121

Each of the many convolutional layer blocks in DenseNet121 is followed by a pooling procedure. The close links between layers guarantee that features learnt in one layer are carried forward, therefore enabling later levels to profit from a more complete collection of features. To guarantee that the model stays efficient despite the great number of connections, the design additionally makes use of compression methods and bottleneck layers. DenseNet121's capacity to solve the vanishing gradient problem—which results from gradients being too tiny during backpropagation, therefore impeding the training of deep networks—is among its main benefits. DenseNet121 maintains a strong gradient flow—even in extremely deep networks—by tying every layer to all next layers. DenseNet121 is thus well suitable for jobs like medical images where the input data includes complicated and hierarchical patterns.

Commonly utilised with transfer learning, DenseNet121 makes use of ImageNet's pre-trained weights much like EfficientNetB0 does. DenseNet121 may be adjusted to certain applications, as identifying chest X-ray images, by adjusting the top layers. DenseNet121 is an excellent

contender for medical image classification as the thick connections and effective feature reuse enable accurate diagnosis by allowing one to recognise subtle and complicated patterns.

3.4.2. Individual CNN

Although pre-trained models such as EfficientNetB0 and DenseNet121 have great benefits, creating a custom CNN model may provide a more customised approach for certain image categorisation chores. Usually created from scratch, custom CNN architectures may be adjusted to the particular features of the current dataset.

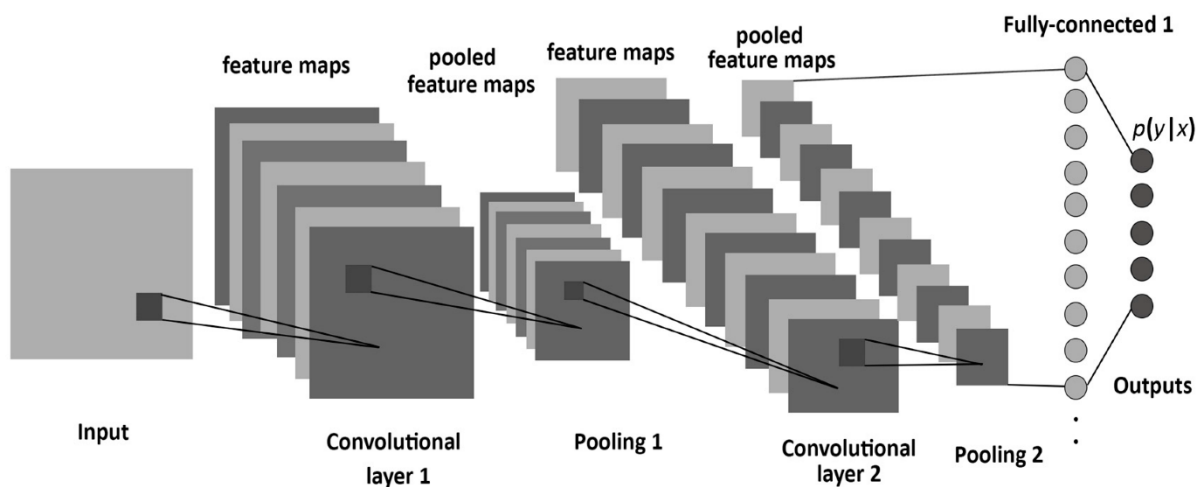


Figure 6 - Architecture of a CNN

Usually consisting of many layers of convolution followed by activation functions like ReLU (Rectified Linear Unit), which adds non-linearity into the network and enables it to learn intricate patterns, the architecture of a custom CNN usually reflects Pooling layers—usually max-pooling—are used to lower the spatial dimensions of the feature maps after the convolutional layers thereby enabling the network to concentrate on the most crucial characteristics and therefore lower computing cost. Usually using one or more fully connected layers after the convolutional and pooling layers have collected pertinent characteristics from the image, the model performs final classification. These fully connected layers provide predictions based on the characteristics learnt by the convolutional layers and combine those features. Typically, in multi-class classification problems, the last layer is a SoftMax layer that generates the probability of every class.

Custom CNN may be rather versatile, letting the design of the network be matched to the particular requirements of the application. Depending on the job difficulty and the volume of input, one may change the number of convolutional layers, filter sizes, and neurones in the

fully connected layers, for instance. Custom CNNs may also be tuned using other methods such as batch normalisation to stabilise the training process or dropout to stop overfit.

3.5. Model Evaluation

Understanding how successfully the models generalise to fresh, unknown data depends critically on assessing their performance on another test set after training. This assessment guarantees that overfitting on the training data does not cause too optimistic performance of the model.

Different evaluation metrics—accuracy, loss, and AUC—Area Under the Curve—are used to assess the models' efficacy. A simple statistic, accuracy gauges the percentage of accurate forecasts among all the model's produced forecasts. High accuracy suggests that the model performs across all classes very well. Conversely, loss measures the degree to which the models' forecasts deviate from the actual values. A reduced loss is a favourable indication of performance as it shows that the model's forecasts are more near to the real labels. For multi-label classification issues, AUC is especially crucial as it gauges the model's capacity to differentiate between positive and negative classes. A greater AUC score indicates that the model can separate the conditions more precisely. Additionally, produced is a classification report to provide more specific analysis of the performance of the model. For every class, this paper offers F1-score, accuracy, and recall. While recall gauges how successfully the model detects all relevant positive events, precision shows the accuracy of the model's positive forecasts. A harmonic mean of accuracy and recall, the F1-score offers a fair assessment of both. These indicators taken together provide a comprehensive picture of the model's performance, particularly in situations where the data could be skewed across classes.

4. Discussion

Three different convolutional neural network (CNN) architectures—EfficientNetB0, DenseNet121, and a custom CNN model—are implemented and evaluated in this study. Each one of these models is built to efficiently categorise multi-label chest X-ray pictures. Let's evaluate every model's architecture and design holistically.

4.1. EfficientNetB0 Architecture Design

Pre-trained on ImageNet, a vast-scale image classification dataset, the model of EfficientNetB0 used in this project is by learning important characteristics from a large collection of generic pictures, this pre-training enables the model to be fine-tuned for the particular job at hand—chest X-ray categorisation. The model is set up using a frozen feature extraction backbone throughout training. Consequently, only the fully connected layers—which provide the final predictions—are trained; the weights of the convolutional layers are not changed.

```
def build_model(image_size = 75, load_previous_weights = True, freeze_cnn = False):
    base_model = EfficientNetB0(include_top=False, input_shape=(image_size, image_size, 3), weights='imagenet')

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(1024, activation='relu')(x)
    x = BatchNormalization()(x)
    x = Dropout(0.5)(x)
    x = Dense(512, activation='relu')(x)
    x = BatchNormalization()(x)
    x = Dropout(0.5)(x)
    predictions = Dense(5, activation='sigmoid')(x)

    model = Model(inputs=base_model.input, outputs=predictions)

    if freeze_cnn:
        for layer in base_model.layers:
            layer.trainable = False

    model.compile(optimizer='adam', loss='categorical_crossentropy',
                  metrics=['accuracy', AUC()])
    return model
```

Figure 7 - EfficientNetB0 Architecture code

The model avoids overfitting and reduces the spatial dimensions of the feature maps by means of GlobalAverage Pooling2D. This pooling layer assists with generalisation and lessens the computational strain, hence benefiting. Following pooling, a sequence of dense layers with ReLU activations and Dropout layers is added to include stochasticity during training thereby preventing overfitting. At last, the output layer uses a sigmoid activation function to do multi-label classification—that is, when the model generates probabilities for every one of the five circumstances rather than just a single class label. For multi-class classification issues, the use of categorical cross-entropy as the loss function is suitable; however, it might be more suited

for exclusive class predictions than for multi-label ones. For every output class, a more appropriate loss function would have been binary cross-entropy with sigmoid activations.

4.2. DenseNet121 Architecture Design

Another potent CNN design with great density between layers is DenseNet121. DenseNet121 is also pre-trained on ImageNet and fine-tuned for chest X-ray classification in the project. DenseNet121's foundation is locked throughout training, same as EfficientNetB0's, with only the fully linked layers changed. Beginning with GlobalAverage Pooling2D to compress spatial dimensions, the model architecture is similar: Dense layers with ReLU activation and Dropout layers for regularisation come next. Training uses the categorical cross-entropy loss function and the sigmoid activation in the last layer likewise does multi-label classification.

```
def build_model_densenet(image_size = 75, load_previous_weights = True, freeze_cnn = False):
    base_model = DenseNet121(include_top=False, input_shape=(image_size, image_size, 3), weights='imagenet')

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(1024, activation='relu')(x)
    x = BatchNormalization()(x)
    x = Dropout(0.5)(x)
    x = Dense(512, activation='relu')(x)
    x = BatchNormalization()(x)
    x = Dropout(0.5)(x)
    predictions = Dense(5, activation='sigmoid')(x)

    model = Model(inputs=base_model.input, outputs=predictions)

    if freeze_cnn:
        for layer in base_model.layers:
            layer.trainable = False

    model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy', AUC()])

    return model
```

Figure 8 - DenseNet121 Architecture code

DenseNet121 and EfficientNetB0 vary notably mainly in their dense connection. DenseNet's dense connections let the model use all past data, which may be very helpful for medical picture subtle feature detection. Because every layer gains from the feature maps produced by all previous layers, DenseNet designs often call for less parameters than conventional CNNs. For medical image classification projects, where knowledge of complex patterns in the picture is essential, DenseNet121 is thus a convincing alternative. DenseNet121's dense connections might, however, make computationally costly in terms of memory and processing time. DenseNet is economical in terms of the number of parameters, but its scalability may be limited by rising memory usage particularly in big datasets or in production applications. Still,

DenseNet121 is a great choice for this work as feature reusing is essential for learning from X-ray pictures.

4.3. Custom CNN Model Architecture Design

Multiple Convolutional layers, Max Pooling layers, and Fully Connected layers define the custom CNN model meant for this project. Starting with a set of Conv2D layers with rising filter sizes to gather varying degrees of feature representations, the architecture The Max Pooling 2D layers serve to shrink the image's spatial dimensions, hence lowering computational complexity and the overfit risk. The model flattens the feature maps after several convolutional and pooling layers and feeds them into fully connected layers.

```
def build_model_cnn(image_size=75, num_classes=5, load_previous_weights=True, freeze_cnn=False):
    model = Sequential()

    model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(image_size, image_size, 3)))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    model.add(Conv2D(64, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    model.add(Conv2D(128, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    model.add(Flatten())

    model.add(Dense(1024, activation='relu'))
    model.add(BatchNormalization())
    model.add(Dropout(0.5))
    model.add(Dense(512, activation='relu'))
    model.add(BatchNormalization())
    model.add(Dropout(0.5))

    model.add(Dense(num_classes, activation='softmax'))

    model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy', AUC()])

    return model
```

Figure 9 - Custom CNN Architecture code

Built from scratch, the custom model offers a point of comparison against pre-trained models such EfficientNetB0 and DenseNet121. This may help one determine if using pre-trained models' transfer learning offers appreciable benefits over a model developed from scratch. The model regularises the network by means of Dropout and ReLU activations in the hidden layers. Usually seen in multi-class classification, the output layer employs a SoftMax activation. Sigmoid activations might be a superior option for multi-label classification, nevertheless, as every class is independent and has own probability. Although the custom CNN has a simple design, for medical image classification it may not perform as well as EfficientNetB0 or DenseNet121. This results from the simplified design and absence of a pre-trained backbone, which may not efficiently capture intricate details. Pre-trained models benefit from using

learnt features from big databases like ImageNet, which usually results in quicker convergence and higher accuracy. Still, the custom model offers a benchmark and helps one compare the performance of more sophisticated models.

Each of the three models—EfficientNetB0, DenseNet121, and the custom CNN—offers unique benefits and compromises. For contexts with limited resources, EfficientNetB0 is an excellent alternative because of its computational economy and capacity to get great accuracy with less parameters. Though it may be more computationally costly, DenseNet121 shines at learning complex patterns from medical pictures with its thick connectedness and feature reusing. Though simpler, the custom CNN model offers a reasonable basis for assessing the performance improvements of more complex designs. Every model has advantages; the decision will rely on the particular trade-offs among accuracy, computing cost, and model complexity.

5. Experimental Results

5.1. EfficientNetB0 Results

Observed during training and evaluation, the EfficientNetB0 model's outputs expose numerous important performance indicators that draw attention to the difficulties of this architecture when applied to the particular job. The model showed poor accuracy in the first epochs; this progressively improved as training went on. With a test accuracy of 58.39%, the model showed some development after training at the conclusion of the 10th epoch. Still, the performance of the model on certain benchmarks like the Area Under the Curve (AUC) and the loss values tempers this accuracy. The test set's AUC was 0.64, which shows space for development especially in terms of class distinction and indicates a modest degree of model performance.

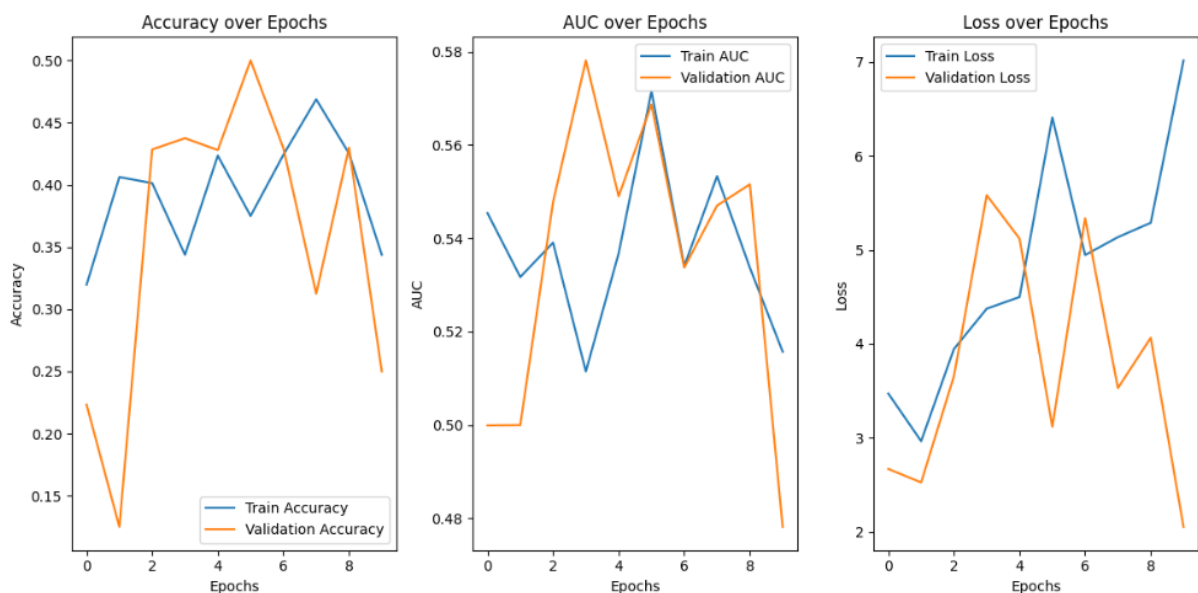


Figure 10 - EfficientNetB0 Evaluation Results

EfficientNetB0's classification report shows regions where the model failed notably even with modest accuracy and AUC. For majority of the classes, the accuracy and recall values were poor, especially for situations like consolidation, oedema, and pleural effusion, where the model exhibited 0 precision and recall, thereby failing to accurately forecast these classes. In medical diagnosis, when missing a condition might have major consequences, the poor recall indicates that the model missed many of the positive occurrences for these conditions, therefore suggesting problems. With scores of 0.62 and 0.72 respectively, cardiomegaly shown somewhat greater accuracy and recall. This implies that EfficientNetB0 was able to

detect cardiomegaly better than other disorders, but the model's shortcomings in accurately forecasting the whole spectrum of diseases meant that overall classification performance remained poor.

5.2. DenseNet121 Results

With accuracy and AUC values somewhat above those of EfficientNet B0, the DenseNet121 model had comparable difficulties. Though the test AUC was 0.58, which is somewhat lower, the model obtained a test accuracy of 58.39%, exactly the same as EfficientNetB0. The model battled to settle on a consistent, optimum solution, so the training and validation losses over the epochs varied. This reflects the difficulty of the work and the likelihood that the model design may not have been completely appropriate for this specific categorisation challenge. Reflecting the very constant validation loss, the validation accuracy at the last epoch was 50%. The model did not demonstrate significant change over time.

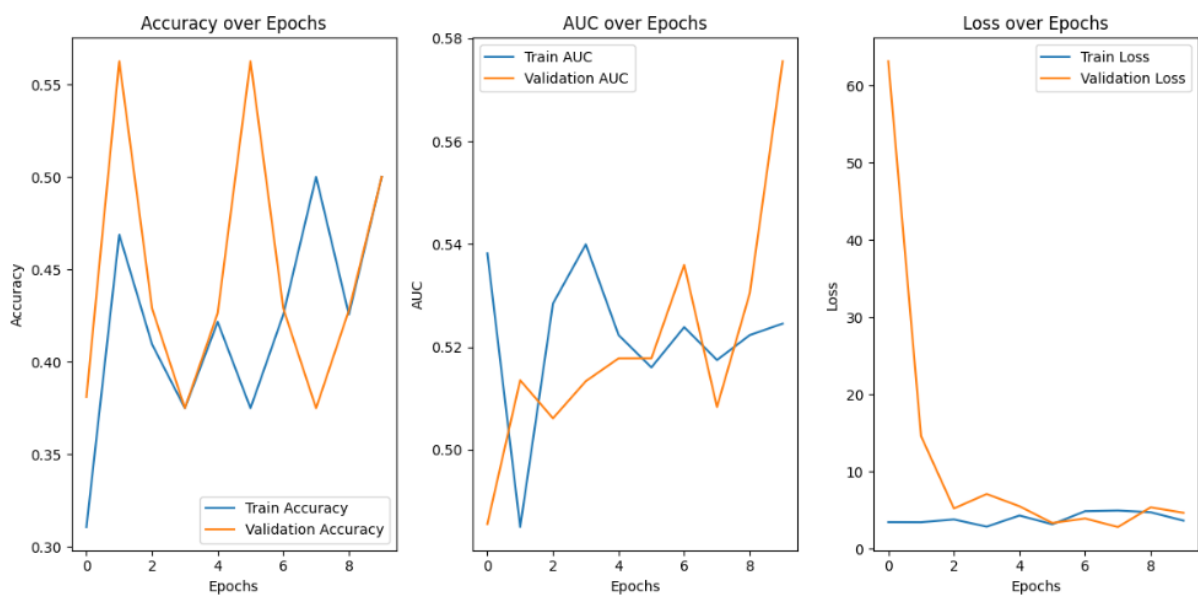


Figure 11 - DenseNet121 Evaluation Results

The classification report of the DenseNet121 model highlights once further its shortcomings. With a recall of 0.76, cardiomegaly once again demonstrated much superior performance; for most other conditions, the precision values were poor. Particularly consolidation, oedema, and pleural effusion were poorly predicted; the model failed to recognise these classes as seen by the 0% recall and accuracy for these circumstances. The absence of identification for important diseases draws attention to DenseNet121's shortcomings in handling unbalanced datasets, wherein certain classes may be under-represented or more challenging to identify.

With a micro average performance that represents its capacity to manage all classes in aggregate, the model had a recall of 0.23, therefore highlighting its difficulties in generalising across the whole dataset.

5.3. Custom CNN Results

With substantially lower overall test accuracy, the findings of the Custom CNN model clearly demonstrated a difference from the pre-trained models. Particularly poorer than the EfficientNetB0 and DenseNet121 models are 13.87% test accuracy and AUC of 0.58. This implies that the bespoke design may be inadequately complicated for the current work or may be less strong in extracting pertinent information from the medical photos. Furthermore, sluggish to converge was the Custom CNN model's training procedure, shown by poor accuracy in the early epochs that hardly changed throughout the course of the training. Based on the poor performance on both accuracy and AUC measures throughout the assessment phase, the last model, after 10 epochs, still showed significant loss and lacked generalising capability.

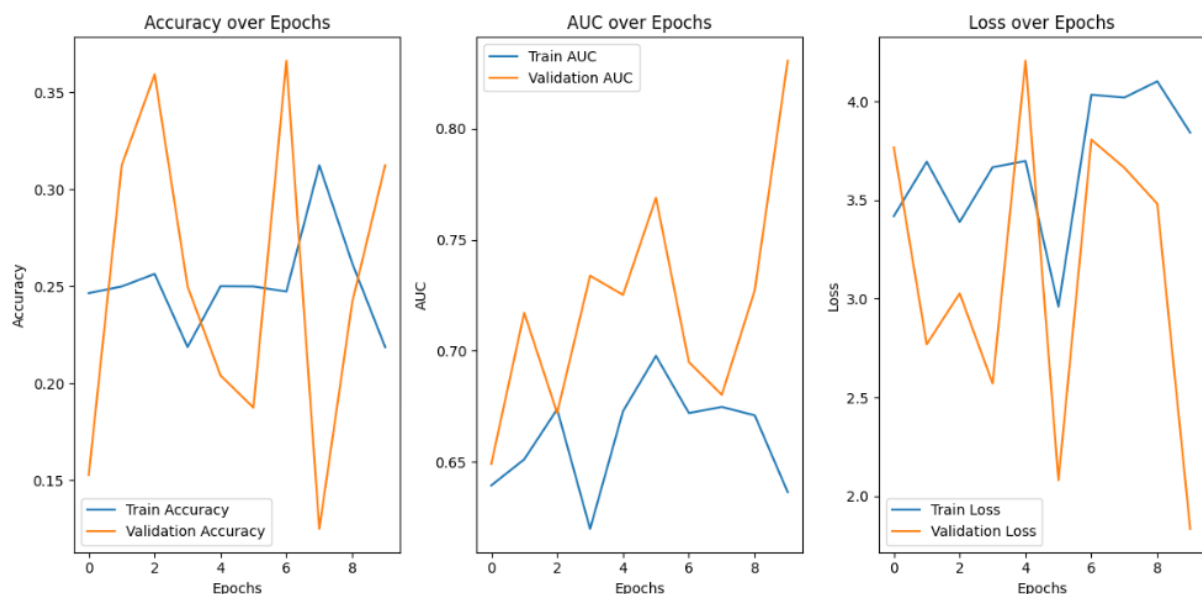


Figure 12 - Custom CNN Evaluation Results

Given the wide disparity in the Custom CNN model's performance on many classes, its classification report is very remarkable. Although the model failed to generalise across the other circumstances, it was very good at detecting atelectasis with a precision of 0.58 and recall of 1.0. Like in the previous models, various disorders like consolidation, oedema, and pleural effusion were not particularly anticipated. Further underlining the model's weak

generalisation capacity, its failure to forecast these scenarios produced 0% accuracy and recall for them. Furthermore, the macro average recall of 0.30 and micro average recall of 0.39 point to major challenges in differentiating between the several diseases, implying either overfitting to some classes or ineffective capture of the intricate patterns in the data.

Each model showed different degrees of performance in the classification test; still, all struggled with class imbalance, poor recall values for certain illnesses, and generalising over the whole range of disorders. Although EfficientNetB0 and DenseNet121 showed rather better accuracy and AUC values than the Custom CNN, all models suffered with precision and recall for most of the classes, suggesting a need for further improvement in the models, maybe through techniques such class weighting, data augmentation, or the use of more sophisticated architectures to address the underlying complexity of medical image classification.

6. Conclusion

This study aimed to assess pre-trained models (EfficientNetB0, DenseNet121) against a bespoke convolutional neural network (CNN) model created from scratch for chest pathology detection in X-ray pictures. The study issue was whether, especially in the diagnosis of chest diseases using a smaller dataset, pre-trained models will surpass models developed from scratch. The findings show a clear variation in the performance of these two varieties of models, therefore supporting the alternative hypothesis (H1).

6.1. Performance of Pre-trained and Custom Models

With regard to test accuracy and Area Under the Curve (AUC), the pre-trained models—EfficientNetB0 and DenseNet121—clearly beat the custom CNN, therefore proving a clear benefit in terms of dataset management. With a test accuracy of 58.39%, both pre-trained models exceeded the custom CNN, which only achieved 13.87%. Moreover, for conditions like cardiomegaly, the pre-trained models demonstrated superior recall and accuracy even if they still battled several pathologies, namely consolidation, oedema, and pleural effusion. Comparatively to the custom CNN, which had a considerably lower AUC of 0.58, the AUC values for EfficientNetB0 and DenseNet121 (0.64 and 0.58, respectively) also revealed that these models managed to acquire more discriminative features from the dataset.

Trained from scratch, the custom CNN model had various difficulties including poor convergence during training, which was evident in its continuous low accuracy and great loss across the epochs. With notably low recall scores for disorders like consolidation and pleural effusion, its inability to generalise successfully across the dataset led in poor performance for most chest pathologies. Though the custom CNN failed with many other classes and had a micro-average recall of only 0.39 even if it could identify atelectasis with great accuracy and recall. This suggests the challenges a scratch-trained model has in faithfully capturing the fundamental patterns of the data, most likely resulting from smaller data size and the lack of pre-learned features from bigger datasets.

Given smaller datasets, the findings of the pre-trained models imply that using models trained on big, varied datasets offers a significant benefit as the transfer of learnt characteristics helps the model generalise better to new, unknown data. Notwithstanding the class imbalance and the difficulties given by the dataset, the pre-trained networks were able to get steady learning

and rather greater accuracy. Conversely, particularly with minimal data, the custom CNN failed to converge and displayed weak generalisation capacity without the advantage of such past experience. Since pre-trained algorithms may use past experience to do challenging tasks with less datasets more efficiently, this supports even more the theory that they identify chest pathologies better.

In conclusion, the findings substantially favour the alternative hypothesis (H1), showing that in identifying chest diseases, pre-trained models far outperform those trained from scratch. This is especially apparent when smaller datasets are employed as the capacity of pre-trained models to transmit acquired characteristics from vast, varied datasets helps them to overcome some of the difficulties presented by small data. The results, however, also highlight the necessity of further developments in medical picture categorisation, especially in managing class imbalance and improving memory for under-represented diseases. Future performance of both pre-trained and bespoke models might benefit from techniques like data augmentation, class weighting, and investigating more complex architectures.

6.2. Future Directions

The main focus of this study will be on raising the general accuracy of the models in identifying chest diseases from X-ray pictures. Data augmentation methods include rotation, flipping, and zooming might help to improve the dataset thus strengthening the model and lowering overfitting. Better feature extraction and context comprehension might also come from investigating cutting-edge designs like Vision Transformers (ViTs) or hybrid models integrating Convolutional Neural Networks (CNNs) with other approaches, including attention mechanisms. Class balancing techniques—such as class weighting or oversampling under-represented classes—can help to solve the apparent imbalance in the present outcomes. Moreover, by measuring prediction uncertainty—which is essential for medical applications—using uncertainty-aware learning approaches could enhance model dependability. Finally, fine-tuning pre-trained models on bigger and more varied datasets could improve classification accuracy as well.

References

- Dong, S. et al. (2021) 'RCoNet: Deformable mutual information maximization and high-order uncertainty-aware learning for robust COVID-19 detection', *IEEE Transactions on Neural Networks and Learning Systems*, 32(8), pp. 3401–3411. doi:10.1109/tnnls.2021.3086570.
- Gour, M. and Jain, S. (2022) 'Uncertainty-aware convolutional neural network for COVID-19 X-ray Images classification', *Computers in Biology and Medicine*, 140, p. 105047. doi:10.1016/j.compbiomed.2021.105047.
- Kuang, Z. et al. (2022) 'Uncertainty-aware deep learning with cross-task supervision for phe segmentation on CT Images', *IEEE Journal of Biomedical and Health Informatics*, 26(6), pp. 2615–2626. doi:10.1109/jbhi.2021.3137603.
- Najdenkoska, I. et al. (2022) 'Uncertainty-aware report generation for chest x-rays by variational topic inference', *Medical Image Analysis*, 82, p. 102603. doi:10.1016/j.media.2022.102603.
- Shamsi, A. et al. (2021) 'An uncertainty-aware transfer learning-based framework for covid-19 diagnosis', *IEEE Transactions on Neural Networks and Learning Systems*, 32(4), pp. 1408–1417. doi:10.1109/tnnls.2021.3054306.
- Whata, A. et al. (2024) 'Uncertainty quantification in multi-class image classification using chest X-ray images of COVID-19 and pneumonia', *Frontiers in Artificial Intelligence*, 7. doi:10.3389/frai.2024.1410841.
- Zahari, R., Cox, J. and Obara, B. (2024) 'Uncertainty-aware image classification on 3D CT Lung', *Computers in Biology and Medicine*, 172, p. 108324. doi:10.1016/j.compbiomed.2024.108324.
- Zhang, M. et al. (2024) 'A new benchmark: Clinical uncertainty and severity aware labeled chest X-ray images with multi-relationship graph learning', *IEEE Transactions on Medical Imaging*, pp. 1–1. doi:10.1109/tmi.2024.3441494.
- Aldhahi, W. and Sull, S. (2023) 'Uncertain-cam: Uncertainty-based ensemble machine voting for improved COVID-19 CXR classification and explainability', *Diagnostics*, 13(3), p. 441. doi:10.3390/diagnostics13030441.

Chegini, M. and Mahlooji Far, A. (2023) 'Uncertainty-aware deep learning-based CAD system for Breast Cancer Classification using ultrasound and mammography images', *Computer Methods in Biomechanics and Biomedical Engineering: Imaging & Visualization*, 12(1). doi:10.1080/21681163.2023.2297983.

Kabir, H.M. et al. (2022) 'Aleatory-aware deep uncertainty quantification for transfer learning', *Computers in Biology and Medicine*, 143, p. 105246. doi:10.1016/j.compbiomed.2022.105246.

Reddy, L.C. et al. (2023) 'An uncertainty-aware deep learning-based model for covid-19 diagnosis', 2023 3rd International Conference on Mobile Networks and Wireless Communications (ICMNBC), pp. 1–6. doi:10.1109/icmnbw60182.2023.10435818.

Yan, S. et al. (2024) 'Diagnose with uncertainty awareness: Diagnostic Uncertainty Encoding Framework for radiology report generation', *Lecture Notes in Computer Science*, pp. 34–44. doi:10.1007/978-3-031-73158-7_4.

Yang, H. et al. (2024) 'Lung nodule segmentation and uncertain region prediction with an uncertainty-aware attention mechanism', *IEEE Transactions on Medical Imaging*, 43(4), pp. 1284–1295. doi:10.1109/tmi.2023.3332944.

Zhan, K. et al. (2023) 'An uncertainty-aware self-training framework with consistency regularization for the multilabel classification of common computed tomography signs in lung nodules', *Quantitative Imaging in Medicine and Surgery*, 13(9), pp. 5536–5554. doi:10.21037/qims-23-40.

Zou, K. et al. (2023) 'A review of uncertainty estimation and its application in medical imaging', *Meta-Radiology*, 1(1), p. 100003. doi:10.1016/j.metrad.2023.100003.

Bacellar, G.C. et al. (2021) COVID-19 chest X-ray image classification using Deep Learning [Preprint]. doi:10.1101/2021.07.15.21260605.

Baltruschat, I.M. et al. (2019) 'Comparison of deep learning approaches for multi-label chest X-ray classification', *Scientific Reports*, 9(1). doi:10.1038/s41598-019-42294-8.

Chen, B. et al. (2020) 'Label co-occurrence learning with graph convolutional networks for multi-label chest X-Ray Image Classification', *IEEE Journal of Biomedical and Health Informatics*, 24(8), pp. 2292–2302. doi:10.1109/jbhi.2020.2967084.

Giełczyk, A. et al. (2022) 'Pre-processing methods in chest X-ray image classification', *PLOS ONE*, 17(4). doi:10.1371/journal.pone.0265949.

Guan, Q. and Huang, Y. (2020) 'Multi-label chest X-ray image classification via category-wise residual attention learning', *Pattern Recognition Letters*, 130, pp. 259–266. doi:10.1016/j.patrec.2018.10.027.

Lakhani, P. (2017) 'Deep convolutional neural networks for endotracheal tube position and X-ray image classification: Challenges and opportunities', *Journal of Digital Imaging*, 30(4), pp. 460–468. doi:10.1007/s10278-017-9980-7.

Madaan, V. et al. (2021) 'XCOVNET: Chest X-ray image classification for COVID-19 early detection using convolutional neural networks', *New Generation Computing*, 39(3–4), pp. 583–597. doi:10.1007/s00354-021-00121-7.

Munusamy, H. et al. (2021) 'FractalCovNet architecture for COVID-19 chest X-ray image classification and CT-Scan Image segmentation', *Biocybernetics and Biomedical Engineering*, 41(3), pp. 1025–1038. doi:10.1016/j.bbe.2021.06.011.

Sharma, A., Rani, S. and Gupta, D. (2020) 'Artificial intelligence-based classification of chest X-ray images into covid-19 and other infectious diseases', *International Journal of Biomedical Imaging*, 2020, pp. 1–10. doi:10.1155/2020/8889023.

Sitaula, C. and Hossain, M.B. (2020) 'Attention-based VGG-16 model for COVID-19 chest X-ray image classification', *Applied Intelligence*, 51(5), pp. 2850–2863. doi:10.1007/s10489-020-02055-x.

Appendices

```
#!/usr/bin/env python
```

```
# coding: utf-8
```

```
# In[1]:
```

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
import os
```

```
import numpy as np
```

```
import pandas as pd
```

```
import datetime
```

```
import matplotlib.pyplot as plt
```

```
import cv2
```

```
from pathlib import Path
```

```
from collections import Counter
```

```
from sklearn.metrics import roc_auc_score
```

```
import tensorflow as tf
```

```
from tensorflow.keras.applications.densenet import DenseNet121
```

```
from tensorflow.keras.applications.inception_v3 import InceptionV3
```

```

from tensorflow.keras.preprocessing import image

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.models import Model

from tensorflow.keras.layers import Dense, Input, GlobalAveragePooling2D, MaxPooling2D,
Flatten, BatchNormalization, Dropout

from tensorflow.keras.callbacks import ModelCheckpoint

from tensorflow.keras import backend as K

get_ipython().run_line_magic('load_ext', 'autoreload')

get_ipython().run_line_magic('autoreload', '')

```

```
# In[2]:
```

```

base_path = Path('C:\\Users\\deepi\\OneDrive\\Desktop\\project')

train_data_path = base_path / 'train'

test_data_path = base_path / 'test'

train_labels_path = base_path / 'train_labels.csv'

def load_data(file_path):

    try:

        return pd.read_csv(file_path)

    except FileNotFoundError:

        print(f"File not found: {file_path}")

```

```
return None
```

```
train_df = load_data(base_path /  
'C:\\Users\\deepi\\OneDrive\\Desktop\\project\\CheXpert-v1.0-small\\train.csv')  
  
valid_df = load_data(base_path /  
'C:\\Users\\deepi\\OneDrive\\Desktop\\project\\CheXpert-v1.0-small\\valid.csv')
```

```
if train_df is not None:
```

```
    print("Training data loaded successfully.")
```

```
else:
```

```
    print("Failed to load training data.")
```

```
if valid_df is not None:
```

```
    print("Validation data loaded successfully.")
```

```
else:
```

```
    print("Failed to load validation data.")
```

```
# In[3]:
```

```
chexnet_targets = ['No Finding', 'Enlarged Cardiomediatinum', 'Cardiomegaly', 'Lung  
Opacity',
```

```
    'Lung Lesion', 'Edema', 'Consolidation', 'Pneumonia', 'Atelectasis',
```

```
'Pneumothorax', 'Pleural Effusion', 'Pleural Other', 'Fracture',  
'Support Devices']
```

```
chexpert_targets = ['Atelectasis', 'Cardiomegaly', 'Consolidation', 'Edema', 'Pleural Effusion']
```

```
# In[4]:
```

```
train_df.head()
```

```
# In[5]:
```

```
u_one_features = ['Atelectasis', 'Edema']
```

```
u_zero_features = ['Cardiomegaly', 'Consolidation', 'Pleural Effusion']
```

```
# In[6]:
```

```
def feature_string(row):
```

```

active_features = [
    feature for feature in u_one_features if row[feature] in [-1, 1]
] + [
    feature for feature in u_zero_features if row[feature] == 1
]

return ';'.join(active_features)

```

In[7]:

```

train_df['train_valid'] = False
valid_df['train_valid'] = True

```

In[8]:

```

train_df['patient'] = train_df['Path'].str.split('/', expand=True)[2]
train_df['study'] = train_df['Path'].str.split('/', expand=True)[3]

valid_df['patient'] = valid_df['Path'].str.split('/', expand=True)[2]

```



```
valid_df['study'] = valid_df['Path'].str.split('/', expand=True)[3]
```

```
# In[9]:
```

```
df = pd.concat([train_df, valid_df])
```

```
df.head()
```

```
# In[10]:
```

```
def feature_string(row):
```

```
    active_features = [
```

```
        feature for feature in u_one_features if row[feature] in [-1, 1]
```

```
    ] + [
```

```
        feature for feature in u_zero_features if row[feature] == 1
```

```
    ]
```

```
    if len(active_features) == 0:
```

```
        return None
```

```
return ';'.join(active_features)
```

```
df['feature_string'] = df.apply(feature_string, axis=1)
```

```
df = df[df['feature_string'].notna()]
```

```
df['feature_string'] = df['feature_string'].apply(lambda x: x.split(";"))
```

```
df.head()
```

```
# In[11]:
```

```
image_paths = df['Path'].head(5)
```

```
image_labels = df['feature_string'].head(5)
```

```
fig, axes = plt.subplots(1, len(image_labels), figsize=(20, 10))
```

```
for ax, path, label in zip(axes, image_paths, image_labels):
```

```
    image = cv2.imread(os.path.join(base_path, path))
```

```
    ax.imshow(image)
```

```
    ax.set_title(label)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# In[12]:
```

```
print(f"Paths: {image_paths}")  
  
print(f"Labels: {image_labels}")
```

```
# In[13]:
```

```
all_labels = [label for chexpert_targets in df['feature_string'] for label in chexpert_targets]  
  
label_counts = Counter(all_labels)  
  
plt.bar(label_counts.keys(), label_counts.values(), alpha=0.5)  
  
plt.xticks(rotation='vertical')  
  
plt.show()
```

```
# In[14]:
```

```
sample_percentage = 0.00  
  
train_df1 = df[~df['train_valid']]  
  
valid_df1 = df[df['train_valid']]  
  
unique_patients = train_df1['patient'].unique()
```

```
sample_mask = np.random.rand(len(unique_patients)) < sample_percentage

sampled_patients = unique_patients[sample_mask]

dev_set = train_df1[train_df1['patient'].isin(sampled_patients)]

train_set = train_df1[~train_df1['patient'].isin(sampled_patients)]

print(f"Validation set size: {valid_df1['Path'].size}")

print(f"Training set size: {train_set['Path'].size}")
```

```
# In[15]:
```

```
train_set = train_set.sample(n=20000, random_state=42)
```

```
# In[16]:
```

```
print(f"Training set size: {train_set['Path'].size}")
```

```
# In[17]:
```

```

datagen= ImageDataGenerator(rescale=1./255,

                             featurewise_center=True,

                             featurewise_std_normalization=True,

                             rotation_range=5,

                             width_shift_range=0.2,

                             height_shift_range=0.2,

                             horizontal_flip=True,

                             validation_split = 0.1)

```

```

test_datagen= ImageDataGenerator(rescale=1./255)

```

```

# In[18]:

```

```

def generate_datasets(image_size = 75):

```

```

    train_generator=datagen.flow_from_dataframe(dataframe=train_set,
    directory=base_path,

                                                x_col="Path", y_col="feature_string", has_ext=True, seed = 42,
    #classes = chexpert_targets,

                                                class_mode="categorical", target_size=(image_size,image_size),
    batch_size=32, subset = "training")

```

```

validation_generator      =      datagen.flow_from_dataframe(dataframe=train_set,
directory=base_path,

                        x_col="Path", y_col="feature_string", has_ext=True, seed = 42,
#classes = chexpert_targets,

                        class_mode="categorical",
target_size=(image_size,image_size), batch_size=32, subset = "validation")

test_generator            =      test_datagen.flow_from_dataframe(dataframe=valid_df1,
directory=base_path,

                        target_size=(image_size,image_size),class_mode='categorical',

                        batch_size=1, shuffle=False,

                        x_col="Path", y_col="feature_string")

return [train_generator,validation_generator,test_generator]

```

In[19]:

```

from tensorflow.keras.applications import EfficientNetB0

from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout,
BatchNormalization

from tensorflow.keras.models import Model

from tensorflow.keras.callbacks import ModelCheckpoint

from tensorflow.keras.metrics import AUC

```

```

import tensorflow as tf

from tensorflow.keras import backend as K


# ## EfficientNet


# In[20]:


def build_model(image_size = 75, load_previous_weights = True, freeze_cnn = False):

    base_model = EfficientNetB0(include_top=False, input_shape=(image_size, image_size, 3),
weights='imagenet')

    x = base_model.output

    x = GlobalAveragePooling2D()(x)

    x = Dense(1024, activation='relu')(x)

    x = BatchNormalization()(x)

    x = Dropout(0.5)(x)

    x = Dense(512, activation='relu')(x)

    x = BatchNormalization()(x)

    x = Dropout(0.5)(x)

    predictions = Dense(5, activation='sigmoid')(x)

    model = Model(inputs=base_model.input, outputs=predictions)

```

```
if freeze_cnn:

    for layer in base_model.layers:

        layer.trainable = False

model.compile(optimizer='adam', loss='categorical_crossentropy',

              metrics=['accuracy', AUC()])

return model
```

```
# In[21]:
```

```
image_size_input = 75

model = build_model(image_size=image_size_input)
```

```
# In[22]:
```

```
datasets = generate_datasets(image_size=image_size_input)

train_generator, validation_generator, test_generator = datasets
```



```
# In[23]:
```

```
model.summary()
```

```
# In[24]:
```

```
epochs = 10
```

```
STEP_SIZE_TRAIN = train_generator.n // train_generator.batch_size
```

```
STEP_SIZE_VALID = validation_generator.n // validation_generator.batch_size
```

```
# In[25]:
```

```
history = model.fit(train_generator, steps_per_epoch=STEP_SIZE_TRAIN,  
                    validation_data=validation_generator,  
                    validation_steps=STEP_SIZE_VALID, epochs=epochs)
```

```
# In[26]:
```

```
STEP_SIZE_TEST = test_generator.n // test_generator.batch_size

test_loss, test_accuracy, test_auc = model.evaluate(test_generator, steps=STEP_SIZE_TEST)

print(f"Test Loss: {test_loss}")

print(f"Test Accuracy: {test_accuracy}")

print(f"Test AUC: {test_auc}")
```

```
# In[27]:
```

```
def plot_training_history(history):

    plt.figure(figsize=(12, 6))

    plt.subplot(1, 3, 1)

    plt.plot(history.history['accuracy'], label='Train Accuracy')

    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

    plt.xlabel('Epochs')

    plt.ylabel('Accuracy')

    plt.legend()

    plt.title('Accuracy over Epochs')

    plt.subplot(1, 3, 2)
```

```
plt.plot(history.history['auc'], label='Train AUC')

plt.plot(history.history['val_auc'], label='Validation AUC')

plt.xlabel('Epochs')

plt.ylabel('AUC')

plt.legend()

plt.title('AUC over Epochs')

plt.subplot(1, 3, 3)

plt.plot(history.history['loss'], label='Train Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel('Epochs')

plt.ylabel('Loss')

plt.legend()

plt.title('Loss over Epochs')

plt.tight_layout()

plt.show()
```

```
# In[28]:
```

```
plot_training_history(history)
```

```
# In[29]:
```

```
test_generator.reset()

predictions = model.predict(test_generator, steps=STEP_SIZE_TEST, verbose=1)

predicted_classes = (predictions > 0.5).astype(int)

true_classes = test_generator.classes
```

```
# In[30]:
```

```
from sklearn.metrics import classification_report

from sklearn.preprocessing import MultiLabelBinarizer

mlb = MultiLabelBinarizer()

true_classes_binary = mlb.fit_transform(true_classes)

predicted_classes_binary = mlb.transform(predicted_classes)

print("Classification Report:")

print(classification_report(true_classes_binary, predicted_classes_binary,
target_names=test_generator.class_indices.keys()))
```

```
# ## DenseNet121
```

```
# In[31]:
```

```
from tensorflow.keras.applications import DenseNet121
```

```
from tensorflow.keras.optimizers import Adam
```

```
def build_model_densenet(image_size = 75, load_previous_weights = True, freeze_cnn = False):
```

```
    base_model = DenseNet121(include_top=False, input_shape=(image_size, image_size, 3),  
weights='imagenet')
```

```
    x = base_model.output
```

```
    x = GlobalAveragePooling2D()(x)
```

```
    x = Dense(1024, activation='relu')(x)
```

```
    x = BatchNormalization()(x)
```

```
    x = Dropout(0.5)(x)
```

```
    x = Dense(512, activation='relu')(x)
```

```
    x = BatchNormalization()(x)
```

```
    x = Dropout(0.5)(x)
```

```
    predictions = Dense(5, activation='sigmoid')(x)
```

```
model = Model(inputs=base_model.input, outputs=predictions)
```

```
if freeze_cnn:
```

```
    for layer in base_model.layers:
```

```
        layer.trainable = False
```

```
    model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy',  
AUC()])
```

```
    return model
```

```
# In[32]:
```

```
image_size_input = 75
```

```
model = build_model_densenet(image_size=image_size_input)
```

```
# In[33]:
```

```
model.summary()
```

```
# In[34]:
```

```
history = model.fit(train_generator, steps_per_epoch=STEP_SIZE_TRAIN,  
                    validation_data=validation_generator,  
                    validation_steps=STEP_SIZE_VALID, epochs=epochs)
```

```
# In[35]:
```

```
STEP_SIZE_TEST = test_generator.n // test_generator.batch_size  
  
test_loss, test_accuracy, test_auc = model.evaluate(test_generator, steps=STEP_SIZE_TEST)  
  
print(f"Test Loss: {test_loss}")  
  
print(f"Test Accuracy: {test_accuracy}")  
  
print(f"Test AUC: {test_auc}")
```

```
# In[36]:
```

```
def plot_training_history(history):
```

```
plt.figure(figsize=(12, 6))

plt.subplot(1, 3, 1)

plt.plot(history.history['accuracy'], label='Train Accuracy')

plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel('Epochs')

plt.ylabel('Accuracy')

plt.legend()

plt.title('Accuracy over Epochs')

plt.subplot(1, 3, 2)

plt.plot(history.history['auc_1'], label='Train AUC')

plt.plot(history.history['val_auc_1'], label='Validation AUC')

plt.xlabel('Epochs')

plt.ylabel('AUC')

plt.legend()

plt.title('AUC over Epochs')

plt.subplot(1, 3, 3)

plt.plot(history.history['loss'], label='Train Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel('Epochs')

plt.ylabel('Loss')

plt.legend()

plt.title('Loss over Epochs')

plt.tight_layout()
```



```
plt.show()
```

```
# In[37]:
```

```
plot_training_history(history)
```

```
# In[38]:
```

```
test_generator.reset()
```

```
predictions = model.predict(test_generator, steps=STEP_SIZE_TEST, verbose=1)
```

```
predicted_classes = (predictions > 0.5).astype(int)
```

```
true_classes = test_generator.classes
```

```
# In[39]:
```

```
mlb = MultiLabelBinarizer()
```

```

true_classes_binary = mlb.fit_transform(true_classes)

predicted_classes_binary = mlb.transform(predicted_classes)


print("Classification Report:")

print(classification_report(true_classes_binary,                predicted_classes_binary,
target_names=test_generator.class_indices.keys()))


# ## Custom CNN


# In[40]:


from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,
BatchNormalization, Dropout


# In[41]:


def build_model_cnn(image_size=75, num_classes=5, load_previous_weights=True,
freeze_cnn=False):

    model = Sequential()

```

```

model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(image_size, image_size, 3)))

model.add(MaxPooling2D(pool_size=(2, 2)))


model.add(Conv2D(64, (3, 3), activation='relu'))

model.add(MaxPooling2D(pool_size=(2, 2)))


model.add(Conv2D(128, (3, 3), activation='relu'))

model.add(MaxPooling2D(pool_size=(2, 2)))


model.add(Flatten())


model.add(Dense(1024, activation='relu'))

model.add(BatchNormalization())

model.add(Dropout(0.5))

model.add(Dense(512, activation='relu'))

model.add(BatchNormalization())

model.add(Dropout(0.5))


model.add(Dense(num_classes, activation='softmax'))


model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy',
AUC()])

```

```
return model
```

```
# In[42]:
```

```
image_size_input = 75
```

```
model = build_model_cnn(image_size=image_size_input)
```

```
# In[43]:
```

```
model.summary()
```

```
# In[44]:
```

```
epochs = 10
```

```
STEP_SIZE_TRAIN = train_generator.n // train_generator.batch_size
```

```
STEP_SIZE_VALID = validation_generator.n // validation_generator.batch_size
```

```
# In[45]:
```

```
history = model.fit(train_generator, steps_per_epoch=STEP_SIZE_TRAIN,  
                    validation_data=validation_generator,  
                    validation_steps=STEP_SIZE_VALID, epochs=epochs)
```

```
# In[50]:
```

```
STEP_SIZE_TEST = test_generator.n // test_generator.batch_size  
  
test_loss, test_accuracy, test_auc = model.evaluate(test_generator, steps=STEP_SIZE_TEST)  
  
print(f"Test Loss: {test_loss}")  
  
print(f"Test Accuracy: {test_accuracy}")  
  
print(f"Test AUC: {test_auc}")
```

```
# In[51]:
```

```
def plot_training_history(history):
```

```
plt.figure(figsize=(12, 6))

plt.subplot(1, 3, 1)

plt.plot(history.history['accuracy'], label='Train Accuracy')

plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel('Epochs')

plt.ylabel('Accuracy')

plt.legend()

plt.title('Accuracy over Epochs')

plt.subplot(1, 3, 2)

plt.plot(history.history['auc_2'], label='Train AUC')

plt.plot(history.history['val_auc_2'], label='Validation AUC')

plt.xlabel('Epochs')

plt.ylabel('AUC')

plt.legend()

plt.title('AUC over Epochs')

plt.subplot(1, 3, 3)

plt.plot(history.history['loss'], label='Train Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel('Epochs')

plt.ylabel('Loss')

plt.legend()

plt.title('Loss over Epochs')

plt.tight_layout()
```

```
plt.show()
```

```
# In[52]:
```

```
plot_training_history(history)
```

```
# In[53]:
```

```
test_generator.reset()
```

```
predictions = model.predict(test_generator, steps=STEP_SIZE_TEST, verbose=1)
```

```
predicted_classes = (predictions > 0.5).astype(int)
```

```
true_classes = test_generator.classes
```

```
# In[54]:
```

```
mlb = MultiLabelBinarizer()
```

```
true_classes_binary = mlb.fit_transform(true_classes)

predicted_classes_binary = mlb.transform(predicted_classes)


print("Classification Report:")

print(classification_report(true_classes_binary,                predicted_classes_binary,
target_names=test_generator.class_indices.keys()))
```